CAMPUS LOST & FOUND SYSTEM

Technical Documentation

Full Project Vision and Development Plan

React + Vite | Java + Spring Boot | PostgreSQL

Build - Test - Understand - Move to the next feature.

1. Project Overview

Campus Lost & Found is a web application designed to help university students and staff report, search for, and recover items that have been lost or found on campus.

Instead of relying on WhatsApp groups, posters, or asking people around campus, users can use one central platform to report an item and search for missing items.

Simple example

A student loses a laptop in the library. They open the system and report:

```
Lost: HP Laptop
Location: Main Library
Date: 5 October
Description: Silver HP laptop with a sticker.
```

Another student finds a laptop and reports it. The system can then help the two users identify a possible match and begin the recovery process.

2. Main Goal

The main goal is to create a simple, organized and secure campus platform for lost and found items.

- Report lost items
- Report found items
- Search for items
- Find possible matches
- Contact the relevant person
- Submit ownership claims
- Return items to their owners
- Track resolved cases

3. Users of the System

Student/User

A normal user can:

- Create an account
- Log in
- Report a lost item
- Report a found item
- Search for items
- View item details
- Submit a claim
- View their reports
- Receive notifications
- Communicate with another user

Administrator

The administrator manages the platform. The admin can:

- Manage users
- View reported items
- Review suspicious reports
- Manage claims
- Remove inappropriate content
- Manage categories
- View system statistics
- Resolve disputes/reports

4. Main Features

4.1 User Registration and Login

Users will create accounts using:

- Name
- Email
- Password

After registering, they can log into the system. The system will use authentication so that users can access their own information securely.

5. Report a Lost Item

A user who has lost something can create a lost-item report. The form will contain:

- Item name
- Category
- Description
- Location where it was last seen
- Date
- Approximate time
- Photo
- Additional information

```
Item: HP Laptop
Type: Lost
Category: Electronics
Location: Main Library
Date: 5 October
Description: Silver HP laptop with a sticker
```

6. Report a Found Item

A person who finds something can report it.

```
Item: HP Laptop
Type: Found
```

```
Category: Electronics
Location: Main Library
Date: 5 October
Description: Silver HP laptop
```

The finder does not necessarily need to know who owns the item. They simply report it so the owner can find it.

7. Search for Items

Users should be able to search through reported items. For example, searching for Laptop could display HP Laptop, Dell Laptop and Lenovo Laptop.

Users can also filter by:

- Lost / Found
- Category
- Location
- Date
- Status

8. Item Categories

The system will initially have categories such as:

- Electronics
- Documents
- Bags
- Clothing
- Keys
- Books
- Jewellery
- Accessories
- Other

More categories can be added later.

9. Item Details

Each item will have its own page. Example:

```
HP Laptop
Status: LOST
Category: Electronics
Location: Main Library
Date: 5 October
Description: Silver HP laptop with a sticker on the back.
Reported by: Student

[ Report / Claim ]
```

Users can view the information and decide whether the item is relevant to them.

10. Possible Item Matching

One of the important features will be matching lost and found reports.

```
Lost report
Black Samsung phone
Lost at Library
5 October

Found report
Black Samsung phone
Found at Library
5 October

Possible Match Found
```

At first, matching can be based on simple information such as:

- Item category
- Item name
- Location
- Date

Later, the matching process can be improved with more advanced matching or AI.

11. Claim an Item

A user who believes they own a found item can submit a claim. They should provide information that proves ownership.

- Why do you believe this is your item?
- Describe something about the item that isn't shown publicly.
- Where did you lose it?

The person who found the item can review the claim. Claim statuses are:

- PENDING
- APPROVED
- REJECTED

12. Item Status

Every item will have a status:

- LOST - The owner is looking for the item.
- FOUND - Someone has reported finding it.
- CLAIMED - Someone has submitted a claim.
- RESOLVED - The item has been successfully returned or the case has been completed.

13. User Dashboard

Every user will have a dashboard. It can show:

```
My Dashboard

Lost Items:       3
Found Items:      2
```

```
Pending Claims:   1
Resolved:         2
```

The user can also see their recent reports.

14. Notifications

The system can notify users about important events. Examples:

- A possible match was found for your lost item.
- Someone submitted a claim for your found item.
- Your claim has been approved.

Notifications can initially be inside the application. Later, email notifications can be added.

15. Messaging

Users may need to communicate about an item. Instead of immediately exposing personal phone numbers, the system can provide internal messaging.

```
User A:
I think this may be my laptop.

User B:
Can you describe something about it that isn't shown in the listing?
```

This can be added after the main system is working.

16. Admin Dashboard

The administrator will have a separate dashboard. It can show:

```
Total Users       1,250
Lost Items          430
Found Items         285
Resolved Cases      190
Pending Claims       35
```

The admin can:

- View users
- Remove users
- View items
- Remove inappropriate reports
- Review claims
- Manage categories
- View statistics
- Handle reported content

17. Security

Security is important because users will have accounts and personal information. The system should include:

- Password hashing
- User authentication
- Authorization

- Role-based access
- Input validation
- Secure API endpoints
- Protection against unauthorized access

There will be two basic roles: USER and ADMIN. A normal user should not be able to access admin functionality.

18. Technology Stack - Frontend

React will be responsible for everything the user sees and interacts with.

- React
- JavaScript
- Vite
- CSS

React will contain:

- Pages
- Forms
- Buttons
- Navigation
- Dashboards
- Item cards
- Search
- Filters

19. Backend

Java + Spring Boot will handle the application's main logic.

- Java
- Spring Boot
- Spring Web
- Spring Data JPA
- Spring Security
- Maven

The backend will provide REST APIs. Example:

```
GET    /api/items
GET    /api/items/{id}
POST   /api/items
PUT    /api/items/{id}
DELETE /api/items/{id}
```

React will communicate with these APIs.

20. Database

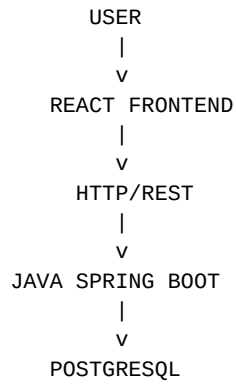
PostgreSQL will store the application's data. Important data will include:

- Users
- Items
- Claims
- Messages
- Notifications
- Categories

```
USER
 |
 |-- ITEMS
 |-- CLAIMS
 |-- MESSAGES
 `-- NOTIFICATIONS
```

21. API Communication

The frontend and backend will communicate using HTTP/REST APIs.



For example, a user clicks Report Item. React sends POST /api/items. Spring Boot receives and validates the information, saves the item in PostgreSQL, and sends a response back to React. React then shows: Item reported successfully.

22. Project Structure

The overall project will have two main applications.

```
CampusLostAndFound/
|
|-- frontend/
|   |-- src/
|   |   |-- components/
|   |   |-- pages/
|   |   |-- services/
|   |   `-- App.jsx
|   `-- package.json
|
`-- backend/
    |-- src/
    |   `-- main/
    |       |-- java/
    |       |   `-- com/campuslostfound/
    |       `-- resources/
    `-- pom.xml
```

23. Development Plan

Phase 1 - Foundation

- React
- Spring Boot
- Git/GitHub
- PostgreSQL
- Frontend/backend connection

Goal: React can communicate with Java.

Phase 2 - Users

- Registration
- Login
- Authentication
- User roles

Goal: Users can securely access the system.

Phase 3 - Lost & Found

- Report lost item
- Report found item
- View items
- Item details
- Edit/delete reports

Goal: The core Lost & Found system works.

Phase 4 - Search

- Search
- Categories
- Filters
- Location
- Date filtering

Goal: Users can easily find relevant reports.

Phase 5 - Claims

- Submit claim
- Review claim
- Approve/reject claim
- Item status updates

Goal: Users can safely recover items.

Phase 6 - Admin

- Admin dashboard
- User management
- Item management
- Claim management
- Reports/statistics

Goal: The system can be properly managed.

Phase 7 - Advanced Features

- Notifications
- Internal messaging
- Email notifications
- Automatic matching
- Image handling
- Advanced search
- Analytics
- AI-assisted matching

Goal: These features are not required for the first version.

24. How We Will Work as a Team

Since two people are building the project, GitHub should be used for collaboration.

Backend

One person works on:

- Java
- Spring Boot
- APIs
- Database
- Authentication
- Backend testing

Frontend

The other works on:

- React
- UI
- Pages
- Forms
- Search
- Dashboard

Both team members should regularly explain their work to each other so that both people understand the complete system.

25. Important Development Rule

Follow this principle: Build -> Test -> Understand -> Move to the next feature. Do not create 20 unfinished features.

```
Registration
    |
    v
Test
    |
    v
Login
    |
    v
Test
    |
    v
Report Item
    |
    v
Test
    |
    v
View Item
    |
    v
Test
```

Each feature should work before moving forward.

26. Final Vision

The final system should allow a student to go from:

```
I lost something.
       |
       v
Report it
       |
       v
Search
       |
       v
Possible match
       |
       v
Submit claim
       |
       v
Verify ownership
       |
       v
Recover item
       |
       v
Case resolved
```

The goal is to make Campus Lost & Found more than just a CRUD project. It should demonstrate understanding of:

- Frontend development

- Backend development
- REST APIs
- Databases
- Authentication
- Authorization
- Software architecture
- Git/GitHub
- Testing
- Team development
- Real-world problem solving

Final Technology Stack

Frontend

- React
- Vite
- JavaScript
- CSS

Backend

- Java
- Spring Boot
- Spring Web
- Spring Data JPA
- Spring Security
- Maven

Database

- PostgreSQL

Development

- Git
- GitHub
- VS Code / IntelliJ IDEA

Communication

- REST API / HTTP
- JSON

Starting point: First make the React frontend and Spring Boot backend communicate. Then build the project feature-by-feature from the foundation upward.